Language Modeling

Introduction to NLP — MSc. DH ENC-PSL

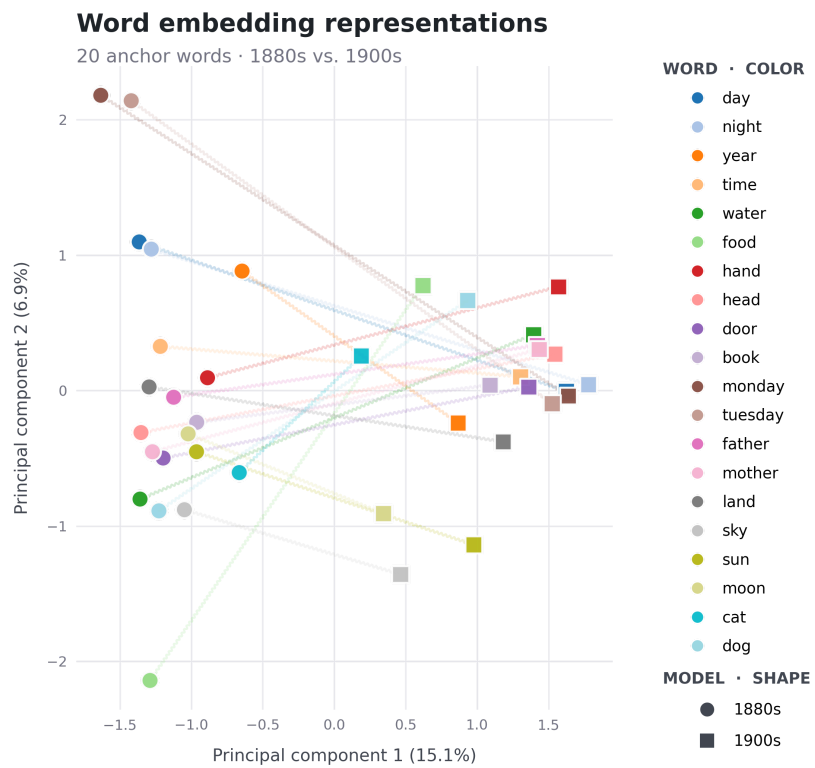
Noé Durandard (noe.durandard@psl.eu)

October 14, 2026

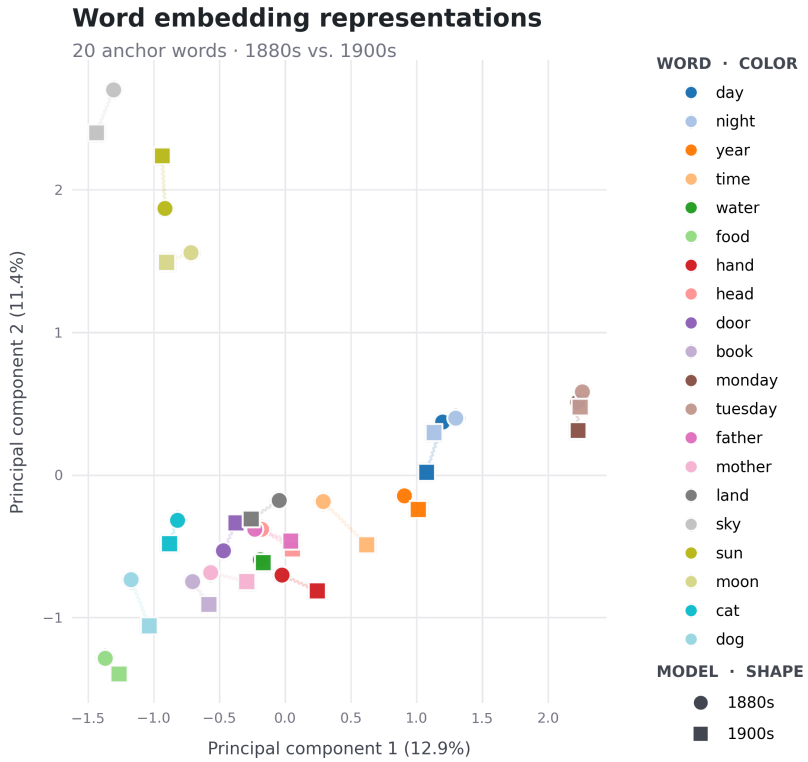
1 Recap

- Tokenization: `raw text` → `discrete tokens` from Vocabulary
  - Different methods: rule-based (whitespaces), subword (BPE)
- Word Vector Representations / Embeddings
  - OHE, word-word co-occurence: long, sparse vectors (dimension: $|\square|$)
  - Word2Vec: short, dense vectors (dimension: $\mathrm{d} \ll |\square|$)
  - Integration of the *distributional hypothesis* to build a geometric *semantic space*
- Semantic Vector Spaces exploration
  - Cosine Similarity: $\operatorname{cossim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \cos(\theta)$
  - Cosine distance: $\operatorname{cosdist}(\mathbf{x}, \mathbf{y}) = 1 - \operatorname{cossim}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = 1 - \cos(\theta)$

Word2Vec applied to: Semantic Shift & Vector-spaces Alignment



Unaligned representations, projected with PCA.



Procrustes aligned representations, projected with PCA.

Table of Contents

- 1 Recap
- 2 Introduction to Language Modeling
- 3 From n-grams to Neural LMs
- 4 Transformer
- 5 Pre-trained Transformer Models
- 6 Hands-on



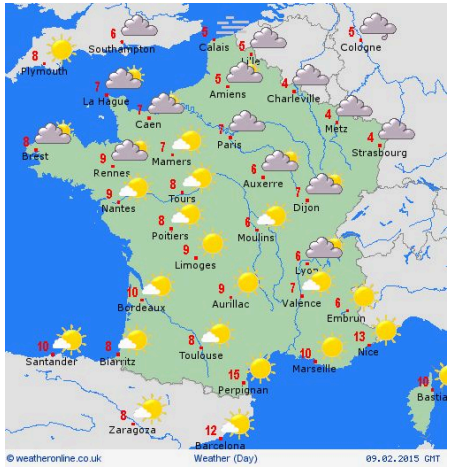
2 Introduction to Language Modeling

2.1 Foundations and Objectives

2.1.1 Modeling

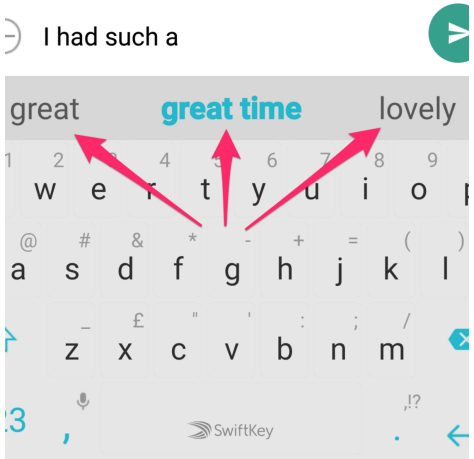
- What do you expect from a **model**?

– simulate behaviour of the real world
– understand which events are in better agreement with the world
– predict the next event given a description of "context" (current state)



- What about **language**?

– Same!
– But what are these *events*?



2.1.2 Modeling Langague

The *linguistic events* used in Language Models are **linguistic unit**: text, sentence, word, token, character, …

Reminder

We defined **tokenization** as the process of breaking down text flow into discret units.

- The *tokens* are the *atomic units* to model language, their choice may depend on the application (context-dependent).

/!\ always make sure all your NLP modules use same notion of *tokens* /!\

2.1.3 Language Modeling Problem

Language Model

A Language Model (LM) estimates the probability of pieces of text.

Given a sequence of text $w_1, w_2, \cdots, w_S$, it answers the question:

What is $P(w_1, w_2, \cdots, w_S)$?

2.1.4 LMs play the role of …

- … a judge of **gramaticality**
  - e.g. *“The player runs.”* vs. *“The player run.”*
- … a judge of **semantic plausability**
  - e.g. *“The teacher spoke.”* vs. *“The blackboard spoke.”*
- … an enforcer of **stylistic consistency**
  - e.g. *“In conclusion, the results substantiate the proposed hypothesis.”* vs. *“In conclusion, the results totally back up the proposed hypothesis.”*
- … a repository of **knowledge (?)**
  - e.g. *“Zinedine Zidane played for Real Madrid.”* vs. *“Zinedine Zidane played for Paris 13 Atletico.”*
  - /!\ Very difficult to guarantee!

2.1.5 *So… Word2Vec?*

→ Was Word2Vec doing Language Modeling all along?

Predictive Objective

Recall the skip-gram algorithm → given a **Center Word** (c), predict its surrounding **Context Words** (o): $P(o \mid c, \theta)$.

It is trained to minimize:

$$-\frac{1}{|\square|}\sum_{c=1}^{|\square|}\sum_{-m\le j\le m, j\neq 0}\log\left(P(w_{c+j}\mid w_c)\right)$$

… but not quite

- Frames language modeling as a *fake task*
  - During training, to extract representations
- No sequential joint probability
  - Does not compute $P(w_1, \ldots, w_S)$; assumes conditional independence and computes local probabilities
  - Cannot evaluate entire sequences

2.1.6 How to get there?

Recall:

Language Model

A Language Model (LM) estimates the probability of pieces of text.

How to compute: $\mathrm{P}(w_1, w_2, \cdots, w_S)$?

- Gain knowledge from corpora.
- **Different formalisms to compute** P

What is the most probable piece of text?

1. *I like Digital Humanities*
2. *Humanities like I Digital*
3. *I likes Digital Humanities*
4. *I bike Digital Humanities*

→ quite intuitive, but how are machines supposed to understand it?

3 From n-grams to Neural LMs

3.1 n-gram models

3.1.1 Reminder: Probabilities

Conditional Probabilities:

$$P(B|A) = P(A,B)/P(A) \Leftrightarrow P(A,B) = P(A)P(B|A)$$

The Chain Rule in general:

$$P(x_1, x_2, x_3, \cdots x_S) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$$
$$\cdots P(x_S|x_1, x_2, x_3, \cdots x_{S-1})$$
$$= \prod_{i=1}^{S} P(x_i|x_1 \cdots x_{i-1})$$

3.1.2 $(n-1)$ Markov Assumption

State *only* depends on $(n-1)$ preceeding states:

$$P(x_i \mid x_1 \cdots x_{i-1}) \approx P(x_i \mid x_{i-n+1} \cdots x_{i-1})$$

$$\Rightarrow P(x_1, x_2, x_3, \cdots x_S)$$

$$\approx \prod_{i=1}^{S} P(x_i \mid x_{i-n+1} \cdots x_{i-1})$$

→ reduces "context" and allows simple (count-based) computation of n-gram probabilities.

The parameters of the model are: $P(x_i \cdots x_{i+n-1})$, which can be estimated on some corpus.

3.1.3 Unigram

$$P(w_1, w_2, \cdots, w_S) = \prod_i P(w_i) = P(w_1)P(w_2) \cdots P(w_S)$$

→ What is the most probable sequence?

$$P(\text{the a the a the a})$$

$$= P(\text{the})P(\text{a})P(\text{the})P(\text{a})P(\text{the})P(\text{a})$$

$$P(\text{the cat sat on the mat})$$

$$= P(\text{the})P(\text{cat})P(\text{sat})P(\text{on})P(\text{the})P(\text{mat})$$

3.1.4 Bigram

$$P(w_1, w_2, \cdots, w_S) = \prod_i P(w_i|w_{i-1}) = P(w_1)P(w_2|w_1) \cdots P(w_S|w_{S-1})$$

→ What is the most probable sequence?

$$P(\text{the a the a the a})$$

$$= P(\text{the})P(\text{a}|\text{the})P(\text{the}|\text{a})P(\text{a}|\text{the})P(\text{the}|\text{a})P(\text{a}|\text{the})$$

$$P(\text{the cat sat on the mat})$$

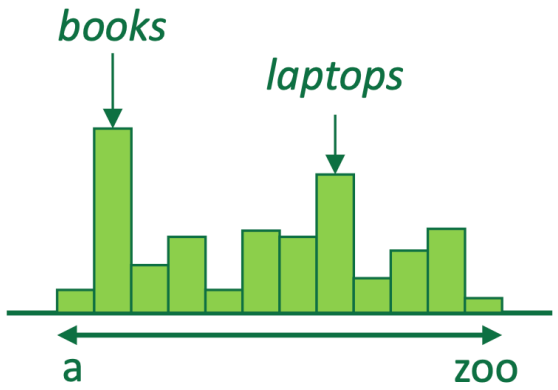
$$= P(\text{the})P(\text{cat}|\text{the})P(\text{sat}|\text{cat})P(\text{on}|\text{sat})P(\text{the}|\text{on})P(\text{mat}|\text{the})$$

3.1.5 Generation with n-gram

- n-gram model *knows* (/has learned) distribution of sizes ≤ n
  - e.g. unigram: P(the), bigram: + P(the cat)
- next-word probability distribution over vocabulary can be inferred:
  $p(w_{S+1} | w_{S-n+1} \cdots w_S)$

The students opend their [?]

→ 4-gram model:
p(?|students opened their)



Sample from distribution:
→ *The students opend their books*

3.1.6 Simple but limited paradigm

Why n-grams?

- Simple and fast.
- Effective for short frequent sequences.
- Clear paradigm introducing important issues for (L)LMs.
  - Training, evaluation, sampling, etc. (we'll come back to this)

Foundational model

All modern neural NLP techniques actually focus on n-grams, estimating various kinds of related probabilities.

Limitations:

- Data sparsity.
  - Scales badly with n.
- Rigid context window.
  - can't handle long dependencies.
- No sense of meaning.

3.2 Neural Language Models

3.2.1 Timeline

100+ years of LMs in 30s:

- **1906:** Markov – Statistical modeling of sequences
- **1948:** Shannon – Information theory and early ideas of statistical prediction of text
- **1980s–1990s:** Back-off and smoothing, probabilistic grammars
- **2003:** *Neural Probabilistic Language Model* (Bengio et al. 2003)
  - First to learn **distributed representations (embeddings)** jointly with next-word prediction
- **2010s:** Recurrent Neural Networks (RNNs), LSTMs, GRUs
  - Allowed *dynamic* context windows
- **2017:** *Attention is All You Need* (Vaswani 2017)
  - self-attention mechansim, drastically reduces training costs
- **2018—:** Pre-trained LLMs + scaling
  - BERT (Devlin et al. 2019)
  - GPT (Radford et al. 2018)

3.2.2 Different paradigm

Language Modeling problem

How to compute $\mathrm{P}(w_1, w_2, \cdots, w_S)$ *or* $\mathrm{P}(w_i | w_1 \cdots w_{i-1})$*?*

Count-based models: rely on explicit co-occurrence counts.

$$\mathrm{P}(w_i | w_1 \cdots w_{i-1}) \approx \frac{\mathrm{C}(w_{i-n+1} \cdots w_{i-1} w_i)}{\mathrm{C}(w_{i-n+1} \cdots w_{i-1})}$$

Neural models: learn a function f_Θ that models NL

$$\mathrm{P}(w_i | w_1 \cdots w_{i-1}) = f_\Theta(w_1, w_2, \cdots, w_{i-1})$$

$\rightarrow$ + based on continuous vector embedding (make semantic emerge).

3.2.3 Neural Probabilistic LM (Bengio et al. 2003)

$$P(w_t | w_{t-n+1:t-1}) = \text{softmax}(g_\theta(e(w_{t-n+1}), \ldots, e(w_{t-1})))$$

- Core ideas:
  - Represent each word as an **embedding vector** (dense representation).
  - Concatenate embeddings of previous $(n-1)$ words.
  - Feed to a feedforward neural network to predict next word probability.

✅ Captures semantic similarity
🚫 One vector per word → doesn't handle polysemy
🚫 Fixed-size context → still limited like n-gram

3.2.4 Recurrent Neural Networks (RNNs)

Motivation

- Fixed-size context of feedforward models is too restrictive.
- Need a model that can handle *arbitrary-length* context.

RNN idea:

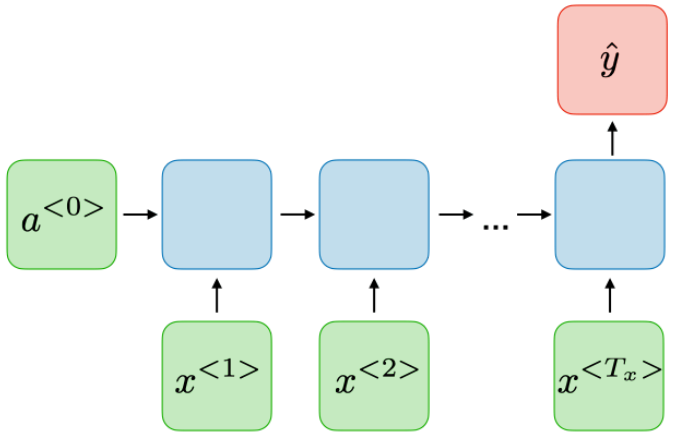
- Maintain a **hidden state** summarizing all past words.
- Update recurrently:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$$

$$P(w_t | w_{1:t-1}) = \text{softmax}(W_{ho}h_t)$$

3.2.4.1 RNN Architecture

- The same weights are used at each time step → efficient and general.
- Capable to capture long-range dependencies.



RNN Architecture (from (Amidi and Amidi 2018)).

Note

Many variants have been developed, e.g. **Long Short-Term Memory (LSTM)** to control information flow (Hochreiter and Schmidhuber 1997) (+later bi-LSTM, context from both sides), Gated Recurent Units (GRUs) simplified approach (Cho et al. 2014). Both mitigate the vanishing gradients arising in traditional RNNs and allow longer dependencies.

3.2.4.2 Limitations of RNNs

- Difficult to capture very long dependencies
  - even with LSTMs.
- Memory bottleneck
  - all context must fit into one hidden vector.
- Slow to train and not prone to parallelization
  - Sequential, recurrent, nature.

→ These challenges motivated **attention mechanisms** and the **Transformer** architecture.

4 Transformer

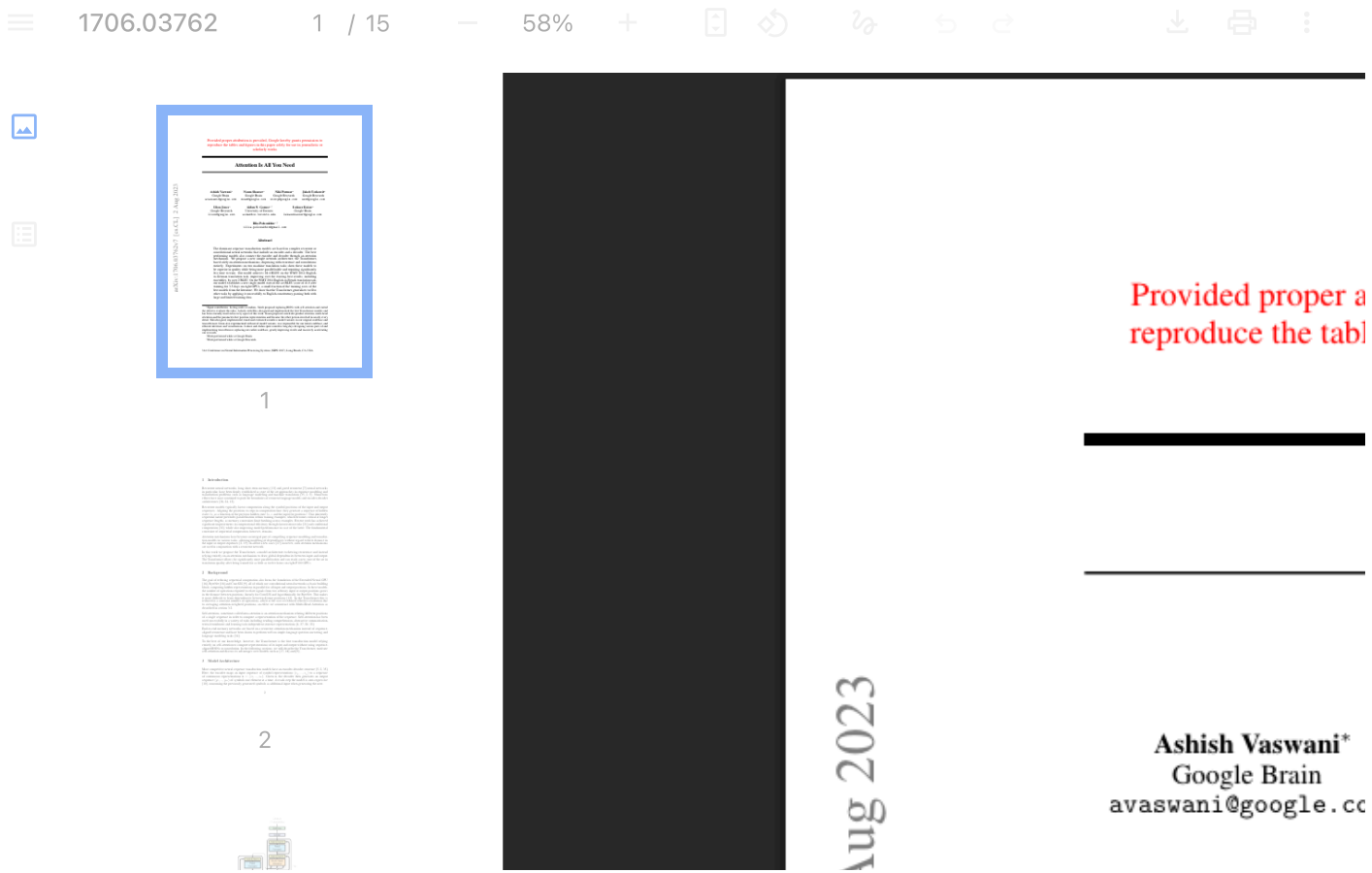
4.1 Motivation

- Need to use more information
  - Take advantage of full context
  - whole sentences available during training
  - why go through sentence word by word?
- Need to improve efficiency
  - Allows parallelization.
  - Based on matrices products.
  - Much faster to train than predecessors.

→ Transformers (with variations) have became standard models for (any) LM tasks.



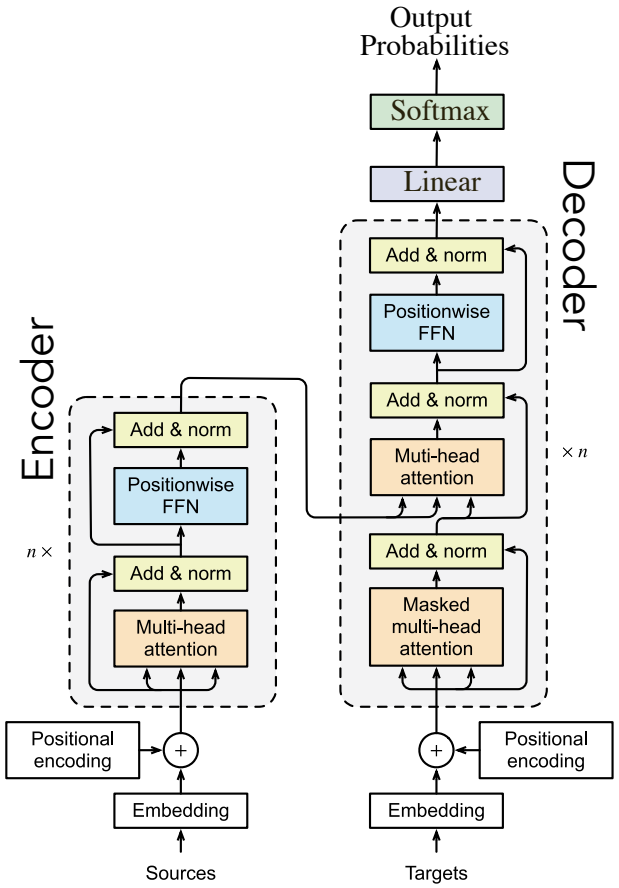
4.2 *Attention is All You Need* (Vaswani 2017)



4.3 The Transformer Architecture

4.3.1 Overview

- Encoder / Decoder Model
  - Building blocks are ≈ similar
- Originally applied to machine translation
  - Foundational for many modern approaches
  - not limited to natural language! (vision, proteins, audio, …)
- Layered approach
  - → model sequentially builds intricate understanding.



Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.3.2 Input: Embedding + Positional encoding

1. Tokenization

- Breaking text into smaller units (from vocabulary).

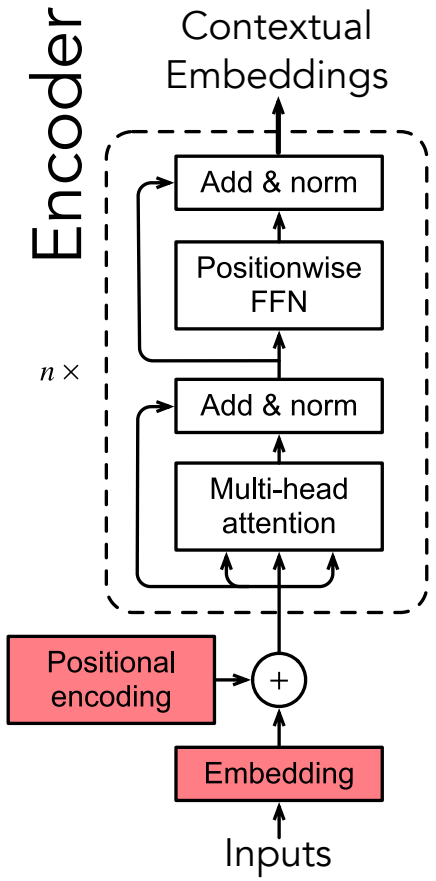
2. Token Embedding

- Associate token with dense vectors (ID → vector).

3. Positional Encoding

- Encode information about token's positions.
- 💡: No direct notion of word order in attention.

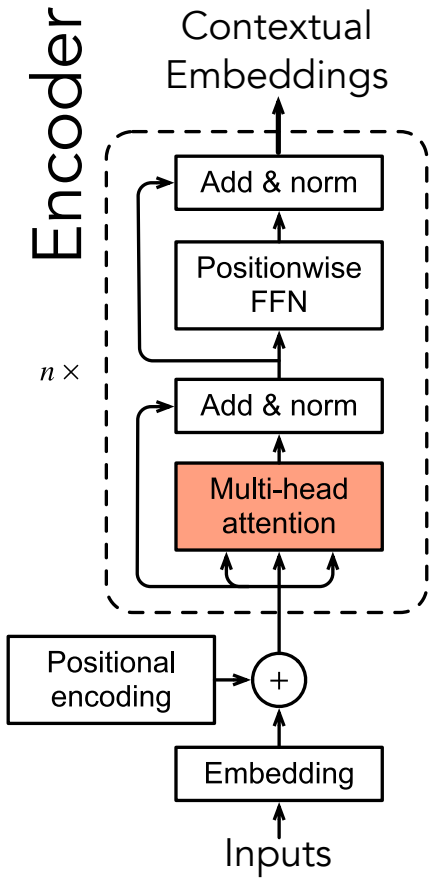
4. Combine: Semantic + Order



Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.3.3 Self-attention Mechanism

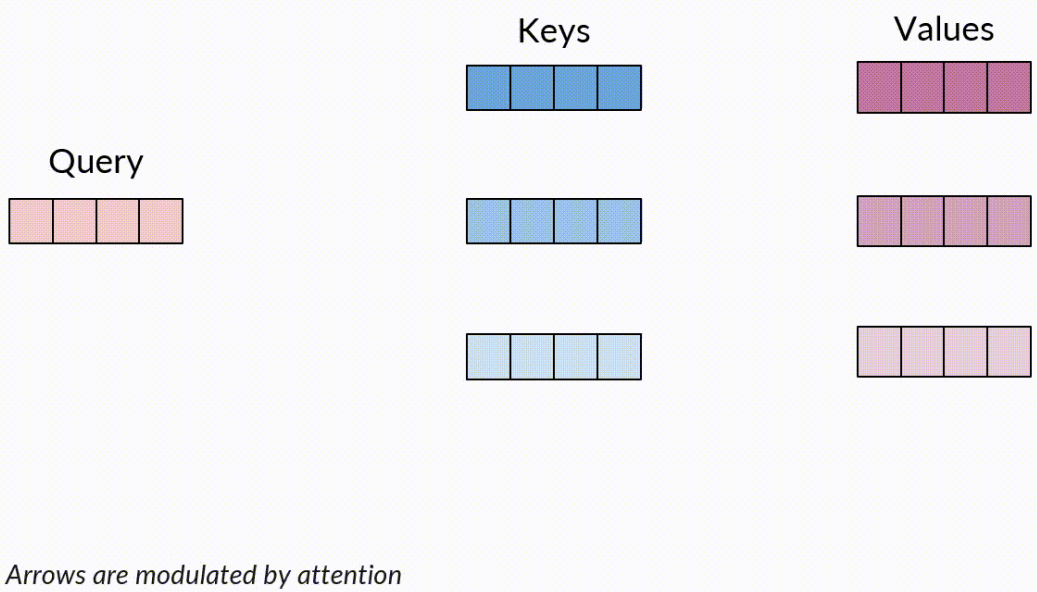
- Capture relationships among tokens in a sequence.
- Split between Multi-Head Attention.



Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.3.3.1 Intuition: Information Look-up

Vaswani et. al 2017, "Attention Is All You Need"



Extracted from (Mittal 2024).

4.3.3.2 Under the hood: Matrices Multiplications

$$\text{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \text{softmax}\left(\frac{\mathrm{QK}^{\mathrm{T}}}{\sqrt{\mathrm{d_k}}}\right)\mathrm{V}$$

where, for a given input $\mathrm{X} \in \mathbb{R}^{\mathrm{N}\times\mathrm{d}}$ and *learnable* projection matrices $\mathrm{W}^{\mathrm{Q}} \in \mathbb{R}^{\mathrm{d}\times\mathrm{d_k}}$, $\mathrm{W}^{\mathrm{K}} \in \mathbb{R}^{\mathrm{d}\times\mathrm{d_k}}$, and $\mathrm{W}^{\mathrm{V}} \in \mathbb{R}^{\mathrm{d}\times\mathrm{d_v}}$:

$$\mathrm{Q} = \mathrm{X}\mathrm{W}^{\mathrm{Q}}, \quad \mathrm{K} = \mathrm{X}\mathrm{W}^{\mathrm{K}}, \quad \mathrm{V} = \mathrm{X}\mathrm{W}^{\mathrm{V}}$$

→ can be parallelized on GPUs!

4.3.3.3 Query-Key-Value

- Q: asking information.
- K: presenting information.

→ QK^T : *attention weights* from one word to another.

- V: giving information.

→ information flow scaled by *attention weights*.

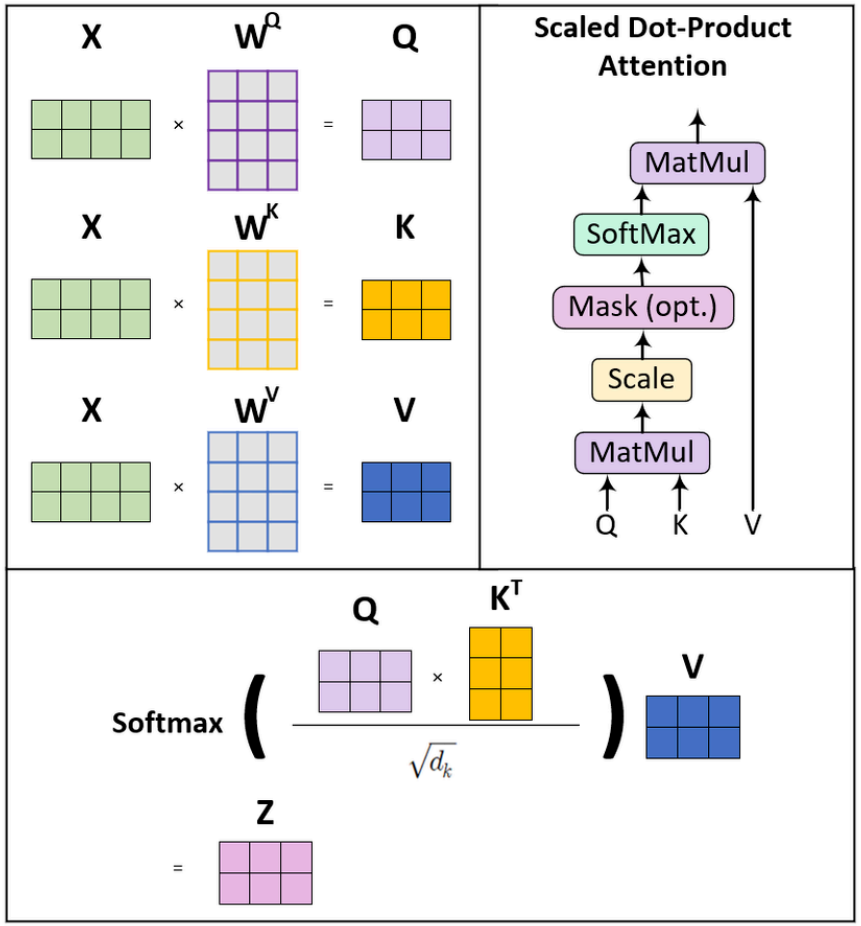
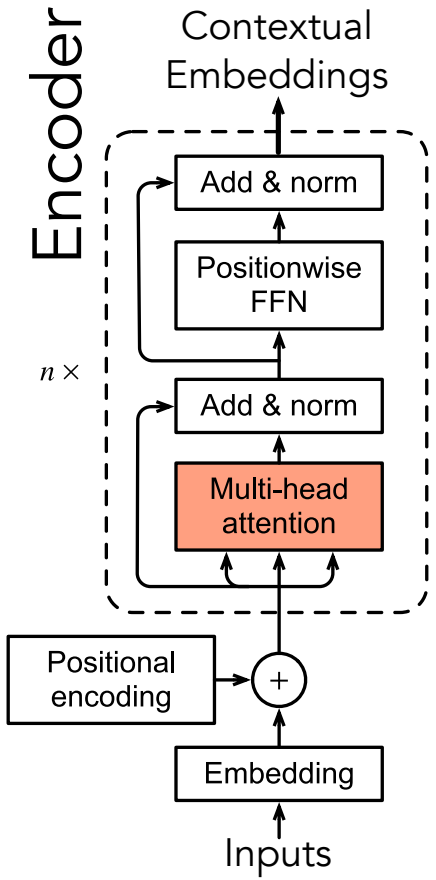


Diagram of the query, key, value, and self-attention mechanism (from (Hwang, Jeong, and Hwang 2025)).

4.3.4 Multi-Head Attention

- **Intuition:** independent “heads” can learn to capture different linguistic relationships.
  - e.g. syntactic, semantic, short-/long- range dependencies, …
- **Implementation:** several attention mechanisms then concatenated.

Think of: “The cat chased the mouse in the garden.”



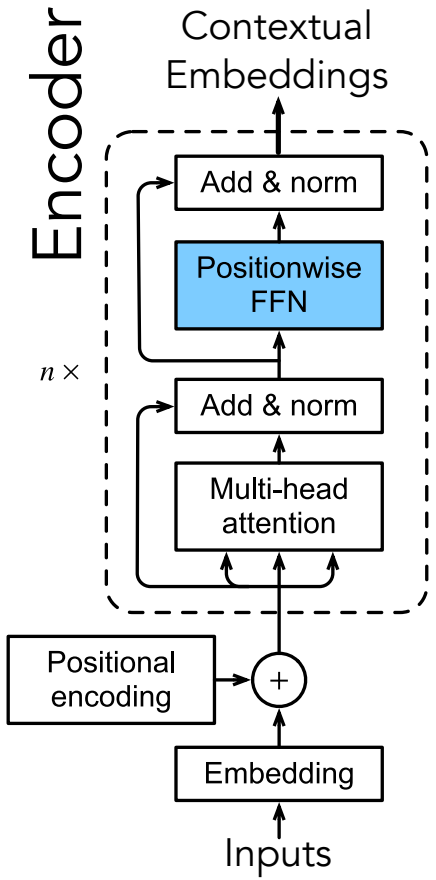
Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.3.5 Feed Forward Network

- **Intuition:** self-attention captures relationships between tokens, FFN refines token-wise representations.
- **Implementation:** two linear layers with ReLU in-between:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)\, W_2 + b_2$$

  - Adds non-linearity.
  - Enrichment through expansion-compression.

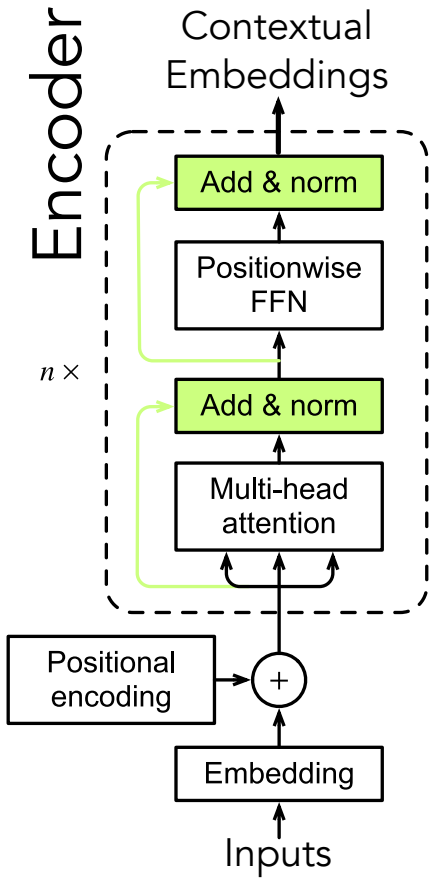


Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.3.6 Residual Connections & Normalization

More technicalities: facilitate information flow and convergence.

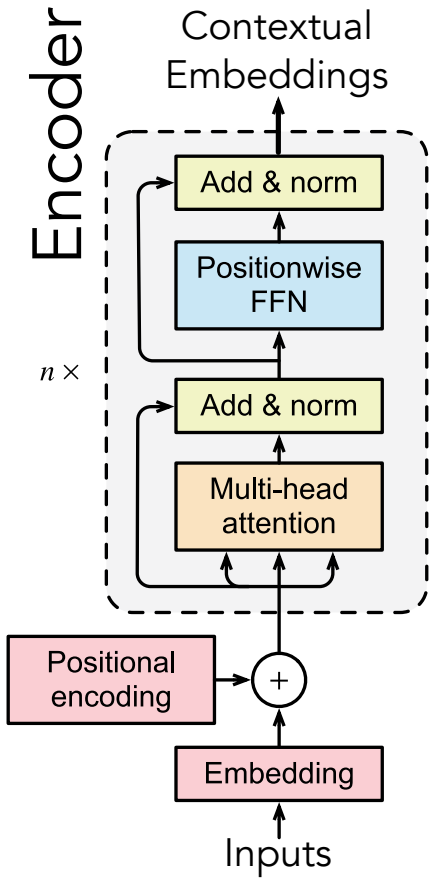
- **Residual Connections**
  - Ease gradient flow.
  - Allow stacking lots of layers.
- **Layer Normalization**
  - Control “flow” to next layer.
  - Improves convergence stability.



Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.3.7 Summary

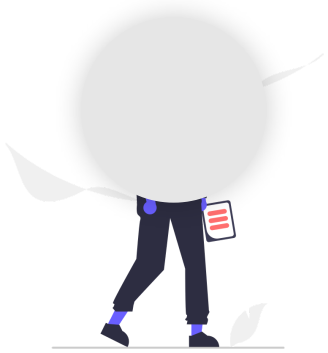
- Positional Encoding:
  - retains information of the word order.
  - enables parallelization and efficient model training.
- Self-Attention Mechanism:
  - global look-up in a sequence.
  - allows direct dependencies.
  - fully parallel.
- Multi-Head Attention:
  - jointly attend to information from different representation subspaces at different positions.
  - provide several perspectives.
  - enhances training stability.
  - effectiveness and optimality discussed in recent literature ((Liu, Liu, and Han 2021), (Mittal et al. 2022)).



Transformer Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4.4 A metaphorical view of Transformer

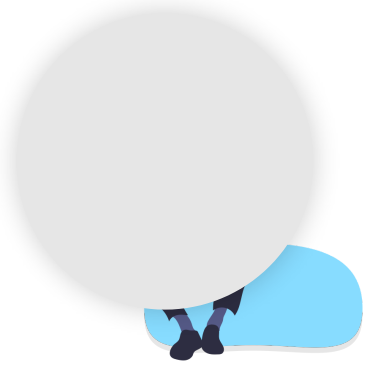
Transformer = Support Groups



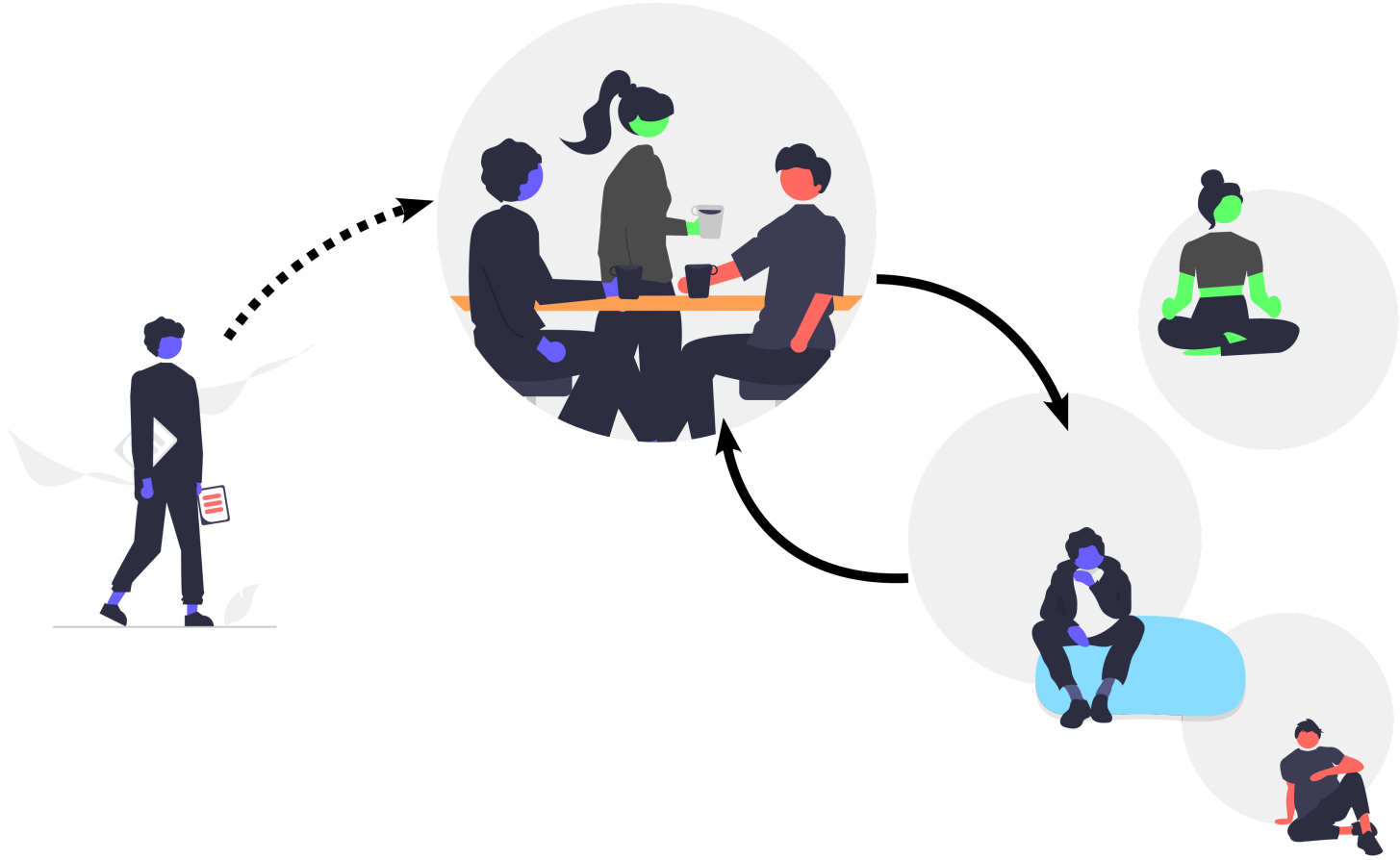
Transformer = Support Groups



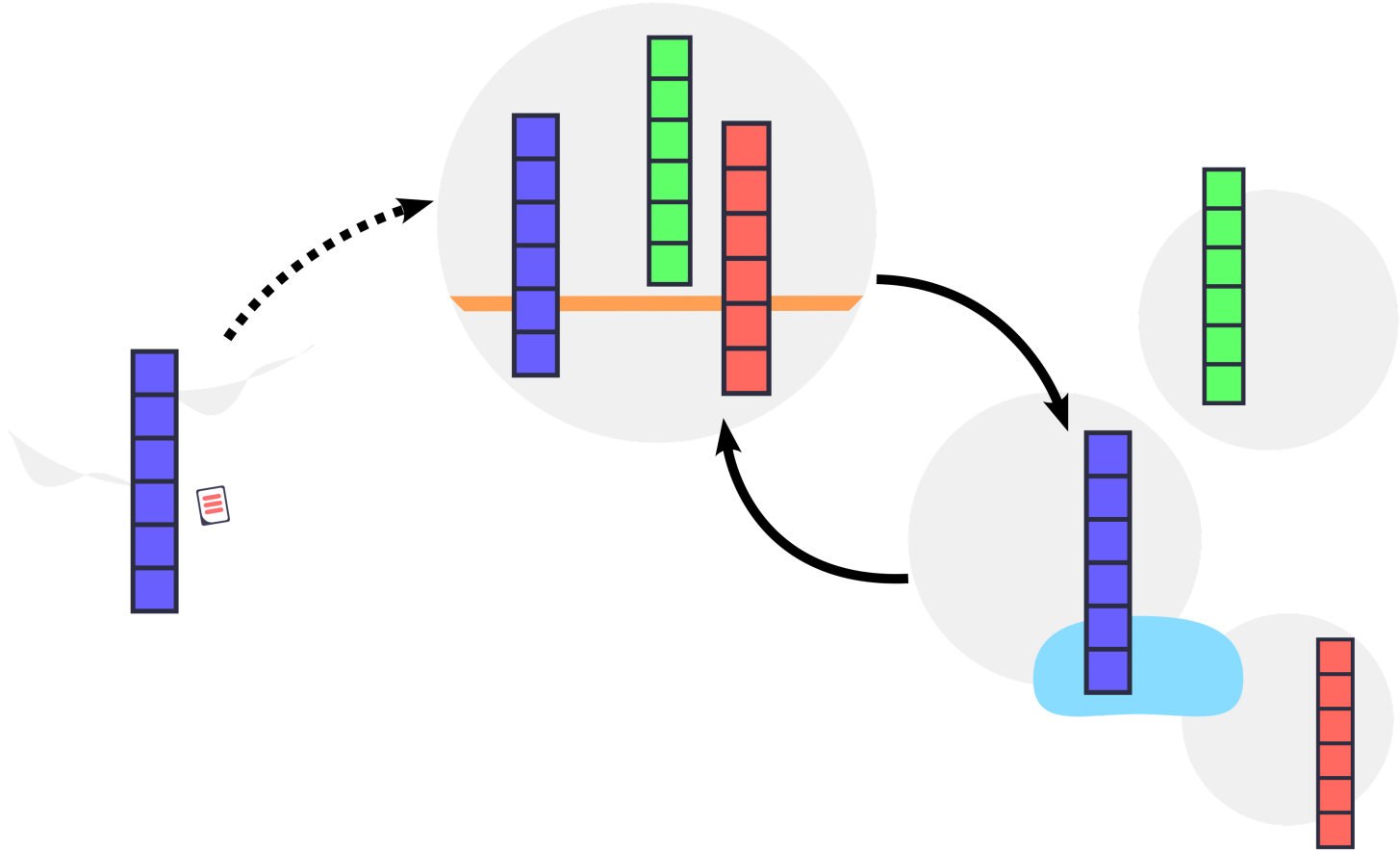
Transformer = Support Groups



Transformer = Support Groups



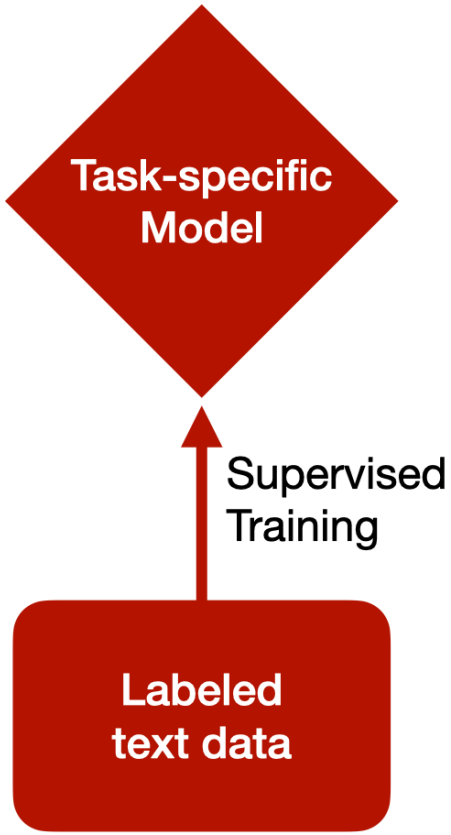
Transformer = Support Groups



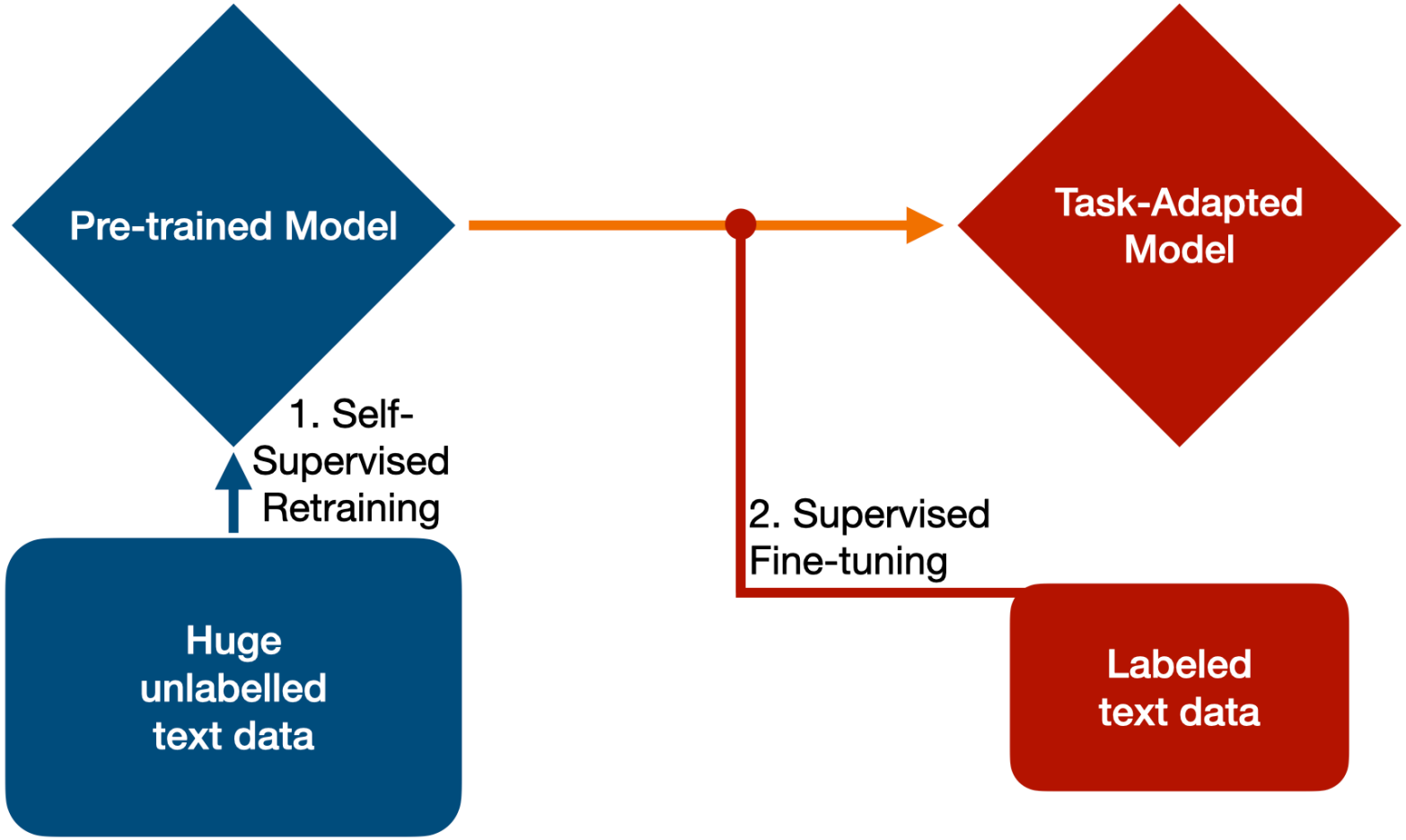
5 Pre-trained Transformer Models

5.1 NLP Pipeline

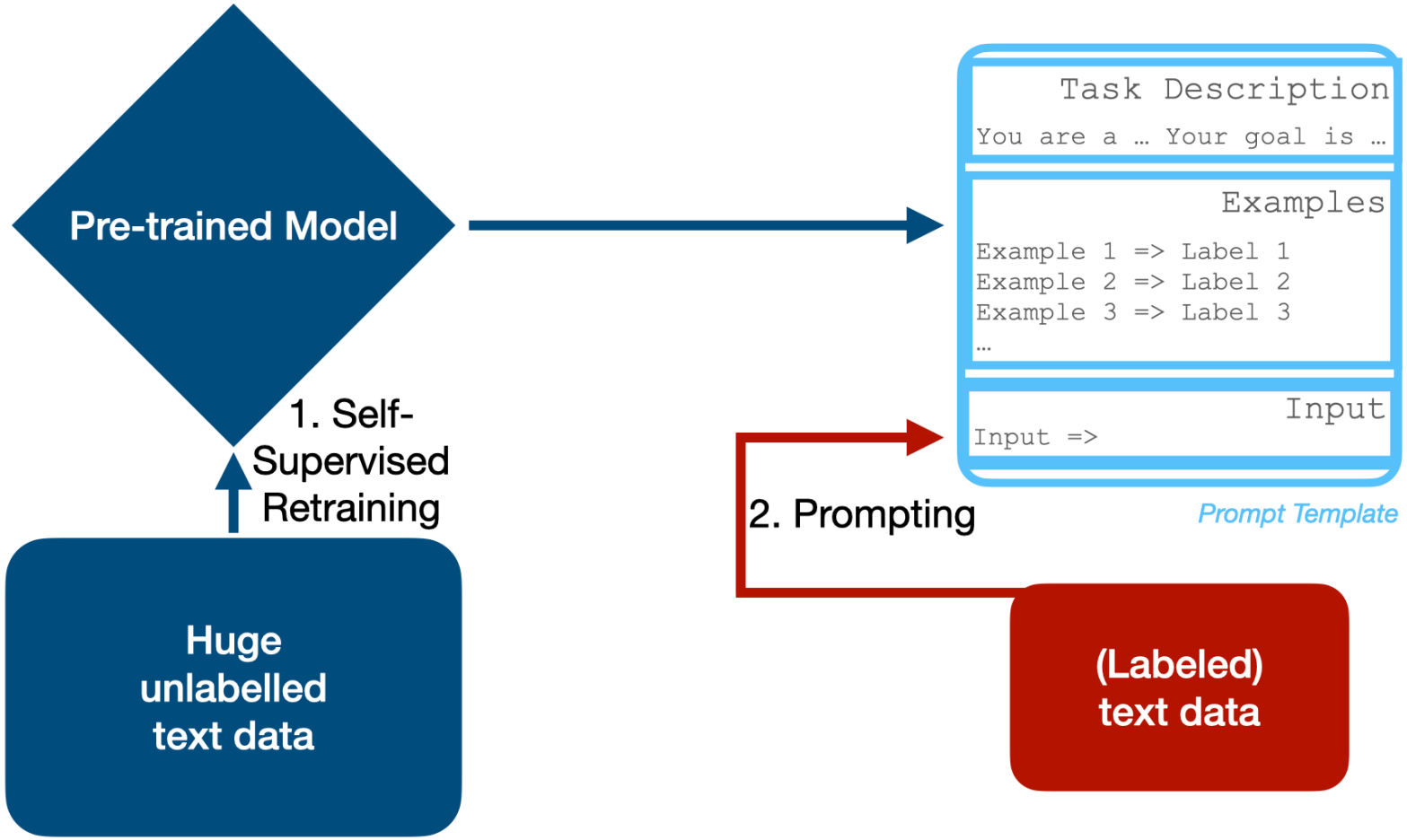
5.1.1 Back in the days



5.1.2 Adapting pre-trained models



5.1.3 Directly prompting



5.2 Taxonomy of pre-trained LMs

5.2.1 Variations in architectures and objectives

	Unidirectional language model	Bidirectional language model	Sequence-to-sequence model
Architecture	Transformer decoder	Transformer encoder	Transformer
Pre-training	Language modeling (2)	Mask language modeling (3)	Sequence-to-sequence learning
Tasks	Language generation	Language understanding	Sequence-to-sequence
Models	GPTs[3,25,26]	BERT,[8] RoBERTa,[17] ALBERT,[14] XLNet,[36] Electra[7]	BART,[15] T5[24]

Overview of pre-trained LM types (from (Li 2022)).

5.2.2 Causal Language Modeling (Decoder)

Predict the **next** token in sequence: $P(w_t \mid w_1, w_2, \ldots, w_{t-1})$

Recently, my friend tapao 20 packets of _____

- Attends to **left-context** only.
- Great for natural language **generation** (… and more?).
- Example: GPTs.

→ We'll come back to this later.

5.2.3 Masked Language Modeling (Encoder)

Predict a **masked** token: $P(w_{MASK} \mid w_{\text{left context}}, w_{\text{right context}})$

Recently, my friend _____ 20 packets of kolo me

- Attends to **left- and right- context** simultaneously (fully bidirectional).
- Great for tasks that require a good **contextual understanding** of an entire sequence.
- Example: BERT-like models.

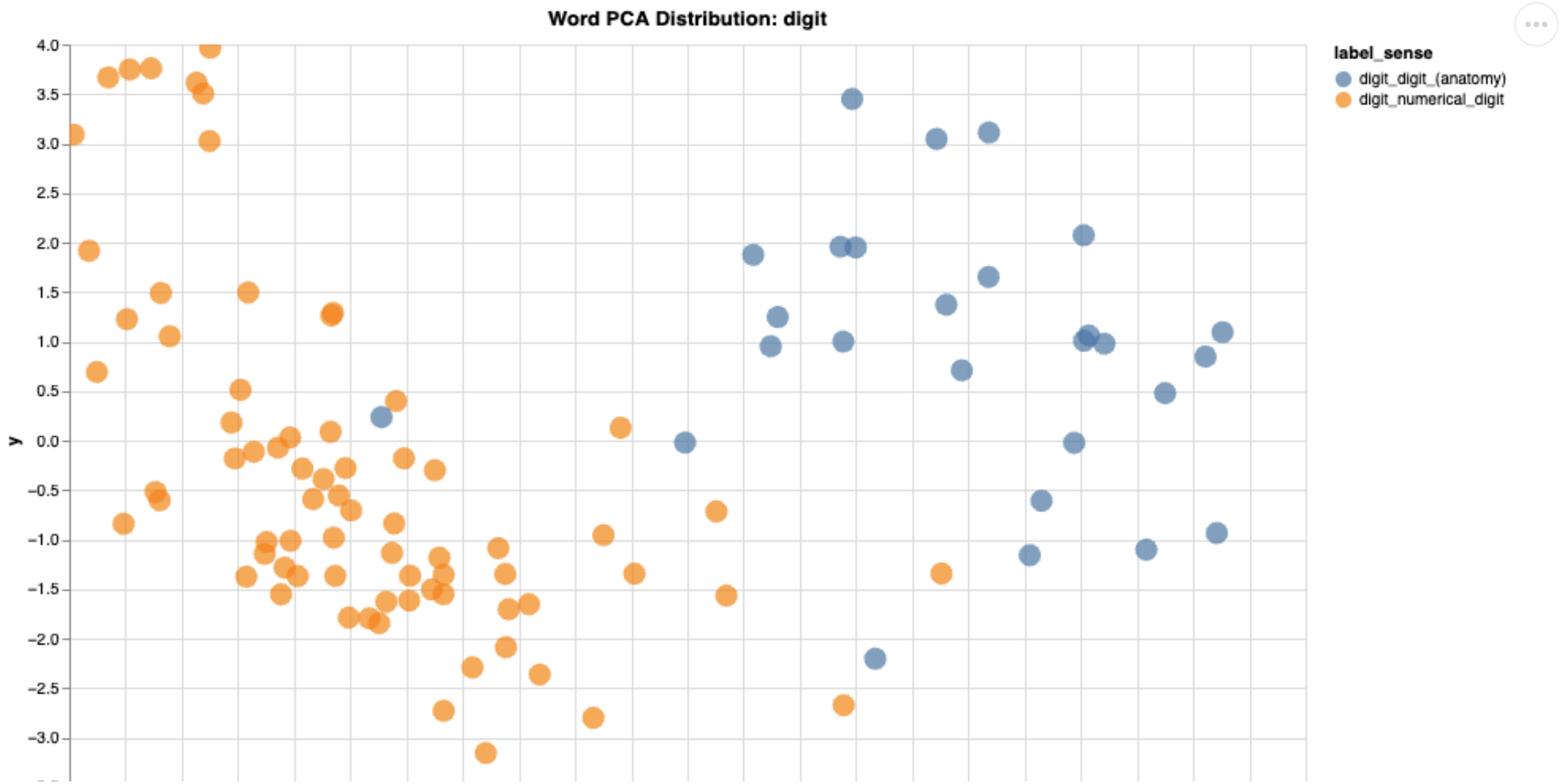
5.3 Focus on BERT (Devlin et al. 2019)



5.3.1 From static to dynamic embeddings

	Word2Vec	BERT
Architecture	Shallow neural network / Matrix factorization	Multi-layer bidirectional Transformer encoder
Contextualization	**Static**: During training; one vector per word type ($\vec{v}_{\text{digit}}$)	**Dynamic**: At inference time; function of the entire sequence / context ($\mathbf{h}^{(l)}_{\text{digit}}$)
Handling Polysemy	❌	✅

Contextualized Embeddings of the token `digit` across occurences in the CoarseWSD-20 dataset (Loureiro et al. 2021):



5.3.2 BERT Specificities

BERT vs other Transformer models

- Original Transformer
  - Encoder & Decoder
  - Sequence-to-Sequence tasks (MT)
- GPT models
  - Decoder-only
  - Text generation

BERT Key Features

- Encoder-only
- True bidirectionality
- Training objectives
  - **Masked Language Modeling (MLM)**: Randomly [MASK] 15% of input tokens; learn to predict ⇒ build rich, bidirectional contextual representations.
  - Next Sentence Prediction (NSP): Predict whether sentence B follows sentence A ⇒ capture relationships between sentences.
  - *NSP much less central in later variants*

→ BERT focuses on natural language understanding (NLU) and representation learning.

5.3.3 BERT Processing

0. Text Input

- `BERT means Bidirectional Encoder Representations from Transformers.`

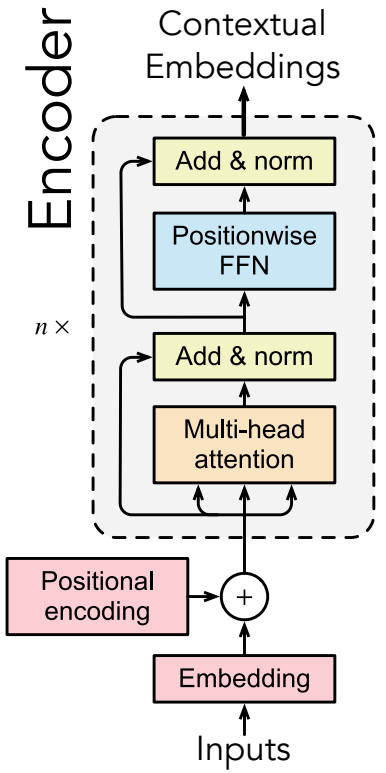
1. Subword (BPE) tokenization (uncased)

- bert means bid ##ire ##ction ##al en##code ##r representations from transformers .

2. Special tokens

- [CLS] bert means bid ##ire ##ction ##al en##code ##r representations from transformers . [SEP]

3. Run through the pre-trained LM →



Transformer Encoder Architecture, based on (Vaswani 2017), reworked from (Zhang et al. 2023).

4. Get outputs:

- Probability distribution, or
- Embeddings shape: $(N_{batch} \times) N_{tokens} \times d_{model}$

5.3.4 BERT Embeddings

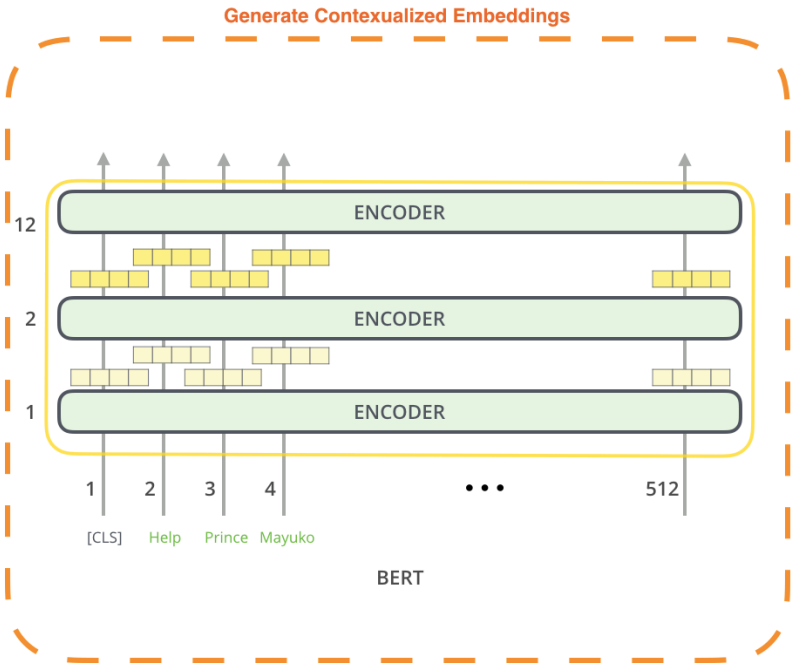
- **Hidden State Representations**

  (e.g., Tenney, Das, and Pavlick 2019):

  - **Lower layers**: Surface syntax, morphology, local patterns
  - **Higher layers**: Semantics, discourse, pragmatic context

- **Choosing the *right* vector**

  - At each layer: $(N_{batch} \times)\ N_{tokens} \times d_{model}$
  - **Layer**: top layer ($\mathbf{h}_i^{(L)}$); other layer ($\mathbf{h}_i^{(l)}$); mean-pooling over layers; concatenation; …
  - **Position**: position of the token; CLS token; mean-pooling over tokens
  - *→ 'Right' choice sually depends on the task, and often results from heuristics or empirical choice.*



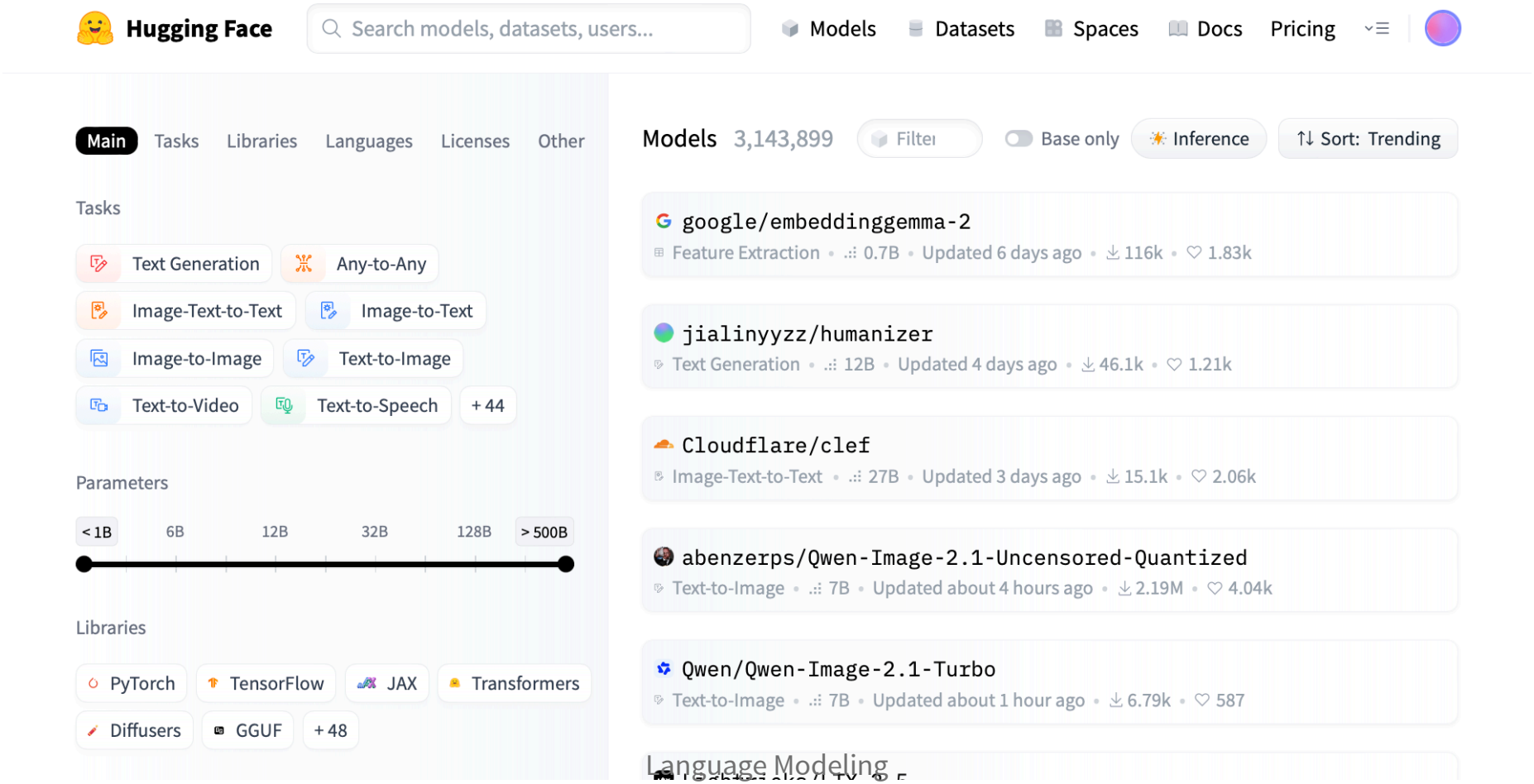
Extracted from *The Illustrated BERT, ELMo, and co.* by Jay Alammar.

5.4 Practical Ecosystem: Hugging Face



5.4.1 Where to find pre-trained LMs?

You can find, access (and share) open-weights LLMs on HuggingFace.



5.4.2 How to use them?

```python
from transformers import AutoTokenizer, AutoModel

model_name = "bert-base-uncased" # choose a model from HF

# Load pre-trained model & tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Extract dynamic contextual embeddings
inputs = tokenizer(
  "The cat didn't cross the street because it was too wide.",
  return_tensors="pt"
)
outputs = model(**inputs)

# Hidden state vector shape: [batch_size, sequence_length, hidden_dim]
last_hidden_states = outputs.last_hidden_state
```

6 Hands-on

1. Turial 3a - Discover BERT
    - Familiarize with BERT, and inspect the inner mechanisms of the model.
2. Tutorial 3b - Word Sense Disambiugation
    - Extract representations from BERT and use them to disambiguate the meaning polysemeous words
3. Hands-on 3 - Semantic Shift with BERT
    - Extract representations from BERT to analyse semantic shifts using data from the *Living With Machines* Initiative.
4. [Bonus] Guided Lab 3 - NER with BERT
    - Adapt a pre-trained BERT-like model to perform Named Entity Recognition (NER) on a corpus of historical newspapers in French (from the *CLEF-HIPE-2020 shared task*).

References

Amidi, Afshine, and Shervine Amidi. 2018. “Recurrent Neural Networks Cheatsheet Star.” *CS 230 - Recurrent Neural Networks Cheatsheet*. https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks#.

Bengio, Yoshua, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. 2003. “A Neural Probabilistic Language Model.” *Journal of Machine Learning Research* 3 (Feb): 1137–55.

Cho, Kyunghyun, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. “On the Properties of Neural Machine Translation: Encoder-Decoder Approaches.” *arXiv Preprint arXiv:1409.1259*.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” https://arxiv.org/abs/1810.04805.

Geva, Mor, Avi Caciularu, Kevin Wang, and Yoav Goldberg. 2022. “Transformer Feed-Forward Layers Build Predictions by Promoting Concepts in the Vocabulary Space.” In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, edited by Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, 30–45. Abu